

Foundations of Gradient boosting

[Gradient boosting]

1.0 Content Block

$\hat{F} = \operatorname{argmin}_F \mathbb{E}_{x, y} [L(y, F(x))]$. $\{\displaystyle \hat{F} = \operatorname{argmin} \limits_{\{F\}} \mathbb{E}_{\{x, y\}} [L(y, F(x))]\}$

2.0 Content Block

Gradient boosting is a machine learning technique based on boosting in a functional space, where the target is pseudo-residuals instead of residuals as in traditional boosting. It gives a prediction model in the form of an ensemble of weak prediction models, i.e., models that make very few assumptions about the data, which are typically simple decision trees. When a decision tree is the weak learner, the resulting algorithm is called gradient-boosted trees; it usually outperforms random forest. As with other boosting methods, a gradient-boosted trees model is built in stages, but it generalizes the other methods by allowing optimization of an arbitrary differentiable loss function.

3.0 Content Block

where $\gamma > 0$ $\{\displaystyle \gamma > 0\}$. For small γ $\{\displaystyle \gamma\}$, this implies that $L(y_i, F_m(x_i)) \leq L(y_i, F_{m-1}(x_i))$ $\{\displaystyle L(y_i, F_m(x_i)) \leq L(y_i, F_{m-1}(x_i))\}$.

4.0 Content Block

Informal introduction (This section follows the exposition by Cheng Li.) Like other boosting methods, gradient boosting combines weak "learners" into a single strong learner iteratively. It is easiest to explain in the least-squares regression setting, where the goal is to teach a model F $\{\displaystyle F\}$ to predict values of the form $\hat{y} = F(x)$ $\{\displaystyle \hat{y} = F(x)\}$ by minimizing the mean squared error $\frac{1}{n} \sum_i (\hat{y}_i - y_i)^2$ $\{\displaystyle \frac{1}{n} \sum_i (\hat{y}_i - y_i)^2\}$, where i $\{\displaystyle i\}$ indexes over some training set of size n $\{\displaystyle n\}$ of actual values of the output variable y $\{\displaystyle y\}$:

5.0 Content Block

Friedman proposes to modify this algorithm so that it chooses a separate optimal value γ_{jm} $\{\displaystyle \gamma_{jm}\}$ for each of the tree's regions, instead of a single γ_m $\{\displaystyle \gamma_m\}$ for the whole tree. He calls the modified algorithm "TreeBoost". The coefficients b_{jm} $\{\displaystyle b_{jm}\}$ from the tree-fitting procedure can be then simply discarded and the model update rule becomes:

6.0 Content Block

Complexity penalty Another useful regularization technique for gradient boosted model is to penalize its complexity. For gradient boosted trees, model complexity can be defined as the proportional number of leaves in the trees. The joint optimization of loss and model complexity corresponds to a post-pruning algorithm to remove branches that fail to reduce the loss by a threshold. Other kinds of regularization such as an ℓ_2 $\{\displaystyle \ell_2\}$ penalty on the leaf values can also be used to avoid overfitting.

7.0 Content Block

for $m \geq 1$ $\{\displaystyle m \geq 1\}$, where $h_m \in H$ $\{\displaystyle h_m \in \mathcal{H}\}$ is a base learner function. Unfortunately, choosing the best function h_m $\{\displaystyle h_m\}$ at each step for an arbitrary loss function L is a computationally infeasible optimization problem in general. Therefore, we restrict our approach to a simplified version of the problem. The idea is to apply a steepest descent step to this minimization problem (functional gradient descent). The basic idea is to find a local minimum of the loss

function by iterating on $F_{m-1}(x)$. In fact, the local maximum-descent direction of the loss function is the negative gradient. Hence, moving a small amount γ such that the linear approximation remains valid:

8.0 Content Block

Regularization Fitting the training set too closely can lead to degradation of the model's generalization ability, that is, its performance on unseen examples. Several so-called regularization techniques reduce this overfitting effect by constraining the fitting procedure. One natural regularization parameter is the number of gradient boosting iterations M (i.e. the number of base models). Increasing M reduces the error on training set, but increases risk of overfitting. An optimal value of M is often selected by monitoring prediction error on a separate validation data set. Another regularization parameter for tree boosting is tree depth. The higher this value the more likely the model will overfit the training data.

9.0 Content Block

Usage Gradient boosting can be used in the field of learning to rank. The commercial web search engines Yahoo and Yandex use variants of gradient boosting in their machine-learned ranking engines. Gradient boosting is also utilized in High Energy Physics in data analysis. At the Large Hadron Collider (LHC), variants of gradient boosting Deep Neural Networks (DNN) were successful in reproducing the results of non-machine learning methods of analysis on datasets used to discover the Higgs boson. Gradient boosting decision tree was also applied in earth and geological studies – for example quality evaluation of sandstone reservoir.

10.0 Content Block

Disadvantages While boosting can increase the accuracy of a base learner, such as a decision tree or linear regression, it sacrifices intelligibility and interpretability. For example, following the path that a decision tree takes to make its decision is trivial and self-explained, but following the paths of hundreds or thousands of trees is much harder. To achieve both performance and interpretability, some model compression techniques allow transforming an XGBoost into a single "born-again" decision tree that approximates the same decision function. Furthermore, its implementation may be more difficult due to the higher computational demand.

11.0 Content Block

where γ_m is the weight at stage m . We are usually given a training set $\{(x_1, y_1), \dots, (x_n, y_n)\}$ of known values of x and corresponding values of y . In accordance with the empirical risk minimization principle, the method tries to find an approximation $\hat{F}(x)$ that minimizes the average value of the loss function on the training set, i.e., minimizes the empirical risk. It does so by starting with a model, consisting of a constant function $F_0(x)$, and incrementally expands it in a greedy fashion: